

COLLEGE OF ENGINEERING AND INFORMATION TECHNOLOGY

LABORATORY MANUAL ACTIVITY

Developing the System Requirements Specification for Your Own Project

Requirements and System Planning

Course	IT 110 – Web Systems and Technologies
Activity	Laboratory Activity No. 5
Output	System Requirements Specification (SRS), requirements-to-feature traceability matrix, and online requirements review
Maximum Score	80 points

Student / Project Information

Block: ____ BSIT 3A SEMQ3	Date: __September 19, 2026____
System Title: RoadWatch – Pothole Detection Using Mobile Dashcam Imagery with a Web-Based Monitoring System	Instructor: Jean Y. Villasin, MIT
Members: RJ Cyrill Jirc L. Monzon; Mae D. Pelipas; Katrice B. Anunciado	

1. Activity Alignment and Purpose

Topic Learning Outcome: Analyze the requirements of the group's semester web-system project and produce a scoped requirements specification aligned with the prescribed architecture and selected development methodology.

Course Outcome Alignment: Supports semester-project planning by translating the community problem, user needs, system boundaries, and project constraints into clear and testable requirements that can guide design, implementation, testing, and review.

Syllabus Assessment: System Requirements Specification; requirements-to-feature matrix; online requirements review.

Purpose of the Activity

Activities 2, 3, and 4 established your project architecture, technology stack, and development methodology. In Activity 5, your team will convert the approved community problem and intended user needs into a controlled requirements baseline. You will define functional and non-functional requirements, user roles and permissions, system scope and boundaries, use cases and/or user stories, feature priorities, acceptance criteria, and project constraints. The resulting SRS and traceability matrix will become the reference for the succeeding design, implementation, database, API, testing, and review activities.

Important distinction: A requirement states what the system must do or what quality/constraint it must satisfy. A feature groups related capabilities that users can recognize. Scope defines what the project will and will not deliver.

Architecture describes how the solution is organized. Do not replace a requirement with a technology name unless the technology itself is a prescribed project constraint.

2. Intended Learning Objectives

- Analyze the community or organizational problem, target users, stakeholders, and desired system outcomes.
- Write clear, atomic, and testable functional requirements using project-specific actions and data.
- Specify measurable non-functional requirements for security, performance, usability, reliability, compatibility, maintainability, and other relevant quality needs.
- Define user roles, permissions, data-access boundaries, and responsibilities using the principle of least privilege.
- Establish the system scope, out-of-scope items, assumptions, dependencies, and project constraints.
- Refine use cases and/or user stories, feature priorities, business rules, and acceptance criteria based on the selected development methodology.
- Assemble a coherent SRS and requirements-to-feature traceability matrix aligned with Flutter Web, the REST API, PostgreSQL, and the team's selected methodology, then prepare the requirements baseline for online review.

3. Activity Specifications

Activity Item	Requirement
Mode	Existing semester-project group; all members must contribute to requirements analysis, refinement, traceability, and review.
Suggested Time	One laboratory session, approximately 3 hours, plus the scheduled semester-project consultation and online requirements review.
Primary Output	One project-specific System Requirements Specification (SRS) and one Requirements-to-Feature Traceability Matrix.
Supporting Output	Problem/user analysis, functional and non-functional requirements worksheets, role-permission matrix, scope and boundary worksheet, use-case/user-story refinement, feature-priority and acceptance-criteria plan, and review notes.
Recommended Tools	Word/Google Docs, diagrams.net/Draw.io or equivalent for use-case diagrams, spreadsheet/table tool for traceability, Git/GitHub, and the team's current project board or documentation workspace.

Activity Item	Requirement
Review Format	Online requirements review through the course platform, with instructor consultation or revision requests when requirements are incomplete, ambiguous, out of scope, or inconsistent with the prescribed architecture.

4. Pre-Laboratory Requirements

- Initial list of target users, stakeholders, major features, data to be collected, and any external services or integrations.
- Available requirement evidence such as stakeholder interview notes, observation results, existing forms/processes, instructor feedback, policy/rule references, or project consultations.
- Access to the project GitHub repository or shared documentation workspace where the SRS, diagrams, review notes, and traceability matrix will be maintained.

5. Part A - Community Problem and Target-User Analysis

Begin with the actual problem before writing features. Requirements should be traceable to a real user need, stakeholder rule, project constraint, or system-quality objective. Complete the project-context worksheet and the target-user/stakeholder analysis before defining detailed requirements.

Project Context Item	Your Project Evidence / Statement
Official system title	RoadWatch – Pothole Detection Using Mobile Dashcam Imagery with a Web-Based Monitoring System
Community / organizational problem	Potholes may go unrecorded during routine travel.
Current process or problem evidence	Group project problem statement; field/interview evidence not yet supplied.
Target community / organization	Road users and authorized pothole monitoring personnel; partner TBC.
Primary users / beneficiaries	Mobile user/driver; monitoring user.
Secondary users / stakeholders	System administrator; road-maintenance stakeholders (if confirmed).
Desired system outcome and success indicator	Detect, save, sync and display potholes only; verify with class-3 tests.

Target User and Stakeholder Analysis Worksheet

User / Stakeholder	Main Goal or Problem	Expected System Interaction	Requirement Source / Evidence
Mobile user / Driver	Detect and record potholes	Start session; hear alert; save/sync records	Approved scope; FR-01–09
Monitoring user	Review pothole records	Open web monitoring	FR-11; FR-14
System administrator	System-level administration	Authorized admin tasks only	Exact permissions: confirm
Not assigned	No additional system role	No access defined	Out of current role scope
Not assigned	No additional system role	No access defined	Out of current role scope
Not assigned	No additional system role	No access defined	Out of current role scope

User / Stakeholder	Main Goal or Problem	Expected System Interaction	Requirement Source / Evidence
Road-maintenance stakeholder	Possible beneficiary	No committed system login	Stakeholder: not verified
Not identified	No external system role	No access defined	Not applicable
Not applicable	No additional role	No access defined	Not applicable
Not applicable	No additional role	No access defined	Not applicable

6. Part B - Functional and Non-Functional Requirements

Write requirements so another team member can implement and test them without guessing. Give each requirement a unique ID. Functional requirements describe system behavior or services. Non-functional requirements describe measurable quality expectations or constraints. Use actual users, data, rules, and project conditions rather than generic statements.

Requirements baseline: v1.0 draft. Verification methods below are planned; test results are not yet recorded.

ID / Type	Requirement Statement	Source / Rationale	Priority	Verification / Acceptance Evidence
FR-01 / F	The mobile user shall start a road-monitoring session using the phone camera.	Driver monitoring need	Must	Start session; show the camera feed.
FR-02 / F	The app shall recognize Pothole (class 3) in camera imagery using the deployed YOLO/ONNX model.	Pothole-only scope	Must	A class-3 sample is accepted.
FR-03 / F	The app shall ignore model classes 0–2 for alert, save, GPS recording, API submission, database storage and web display.	Pothole-only rule	Must	Crack-only test produces zero downstream actions.
FR-04 / F	The app shall play an audible alert for an accepted pothole detection.	Driver awareness	Must	Sound plays for accepted class-3 detection.
FR-05 / F	The app shall attach the phone's GPS coordinates to a pothole record when GPS is available.	Location evidence	Must	Saved latitude/longitude match test location.
FR-06 / F	The app shall save each accepted pothole detection as a detection record.	Record keeping	Must	Saved detection is present in local records.
FR-07 / F	When offline, the app shall queue eligible pothole records locally.	Intermittent network	Must	A new accepted record is marked pending offline.
FR-08 / F	When connectivity returns, the app shall retry synchronization of queued records.	Offline continuity	Must	Pending record is retrieved after reconnection.
FR-09 / F	The app shall submit eligible pothole records through the REST API.	Client/server workflow	Must	An accepted request reaches FastAPI.
FR-10 / F	The API shall store valid pothole detection records in PostgreSQL.	Record persistence	Must	Corresponding record exists in the database.
FR-11 / F	The web monitoring system shall display stored pothole records.	Monitoring need	Must	A stored record appears in the web monitor.
FR-12 / F	The API shall reject invalid and non-pothole detection payloads.	Data integrity	Must	Crack/invalid payload is not stored.

ID / Type	Requirement Statement	Source / Rationale	Priority	Verification / Acceptance Evidence
FR-13 / F	The system shall prevent retries of the same queued record from creating duplicate database rows.	Sync integrity	Must	Retry same record ID; database has one row.
FR-14 / F	The monitoring user shall be able to review available pothole records through the web monitoring system.	Monitoring need	Must	Monitoring user opens available records.
NFR-01 / N	On the declared test phone, audible alert should begin within 2 seconds after an accepted pothole detection.	Alert timing; proposed target	Should	Time 10 accepted detections; record measurements.
NFR-02 / N	A submitted detection confidence shall be numeric and within 0–1 inclusive.	Value validity	Must	Test 0, 1, out-of-range and nonnumeric values.
NFR-03 / N	Queued pothole records shall remain available after the app closes and reopens while offline.	Reliability	Must	Queue survives offline app restart.
NFR-04 / N	When available, saved latitude shall be within –90 to 90 and longitude within –180 to 180 and shall not be swapped.	Coordinate integrity	Must	Test known coordinates and bounds.
NFR-05 / N	The app shall run on the team's declared Android test device and OS version.	Compatibility	Must	Record Android device/OS and test result.
NFR-06 / N	Mobile and web clients shall access stored detection data through the REST API, not directly from the database.	Layer separation	Must	Check data flow/configuration.
NFR-07 / N	The app shall give understandable status/error feedback for camera denial, GPS unavailability and failed submission without falsely showing success.	Usability and failures	Must	Simulate failures and inspect on-screen status.
NFR-08 / N	The Android app shall not connect directly to PostgreSQL or contain PostgreSQL connection credentials.	Data security	Must	Inspect app configuration and network route.

Requirement-writing rule

Prefer a testable form such as “The system shall ...”. Keep one requirement per statement. Replace vague words such as fast, secure, easy, user-friendly, reliable, or real-time with measurable conditions whenever the project can define them. A functional requirement should identify the actor or trigger, required behavior, important data or rule, and expected outcome. A non-functional requirement should state the quality attribute, condition, target or threshold, and how it can be verified.

7. Part C - User Roles and Permissions

Define the actual access level of every system role. The role-permission matrix must agree with the use cases/user stories and with the authentication and authorization controls shown in the architecture. Do not give every user full access simply to simplify implementation.

User Role	Purpose / Main Tasks	Allowed Actions	Restricted / Prohibited Actions	Data Access Scope	Authentication / Permission Rule
Mobile user / Driver	Run road monitoring	Detect; save; sync	No DB/admin access	Own app records	Phone permissions
Monitoring user	Review records	View potholes on web	No model/admin changes	Authorized view	Access limited to role
System administrator	System-level settings	Admin tasks: confirm	No unsupported actions	Admin scope: confirm	Authorized admin only
Not assigned	No other role	None	No access	None	Not applicable
Not assigned	No other role	None	No access	None	Not applicable
External / limited	No external login	None committed	No direct access	None	Not applicable

Role and access review checks:

- Assign only the permissions required for each role to complete its responsibilities.
- Identify which records a user may view, create, edit, approve, delete, export, or administer.
- Separate sensitive or administrative actions from ordinary user actions and define where authorization must be enforced.
- Ensure each permission is reflected in at least one requirement, use case/user story, or explicit out-of-scope rule.

8. Part D - System Scope, Boundaries, Assumptions, and Constraints

Selected methodology from Activity 4:

Agile / Scrum

Prescribed architecture: Flutter Web -> REST API -> PostgreSQL

System objective / intended outcome (2-3 sentences):

RoadWatch detects potholes, alerts drivers and records available GPS. Eligible pothole records sync via the API and appear in the web monitor.

Scope and Boundary Worksheet

For each major module or feature area, state what the semester project will deliver, what will remain outside the project boundary, who uses it, what data it needs, and which dependencies or constraints affect implementation.

Module / Area	In Scope	Out of Scope	User(s)	Main Data	Dependency / External Service	Constraint / Rationale
Mobile detection	Phone camera + class 3	Crack alerts/records	Driver	Frame/class	Android, YOLO/ONNX	Pothole only
Alert and GPS	Sound; available GPS	Crack GPS/audio	Driver	Class; GPS	Phone permissions	GPS optional
Offline queue	Save pending; retry	Non-pothole queue	Driver	Pending record	Device storage	Network varies
REST API	Submit/validate class 3	Direct mobile DB	Driver	Detection payload	FastAPI / network	Validation required

Module / Area	In Scope	Out of Scope	User(s)	Main Data	Dependency / External Service	Constraint / Rationale
PostgreSQL	Persist valid potholes	Non-pothole rows	System	Pothole row	API; PostgreSQL	Server only
Web review	Display stored potholes	Crack reporting	Monitor	Pothole records	Web → API	Authorized only
Admin scope	System administration	Unapproved extra tools	Admin	Settings: confirm	Approved credentials	Confirm actions
Data/privacy	Limit location access	Public GPS release	All roles	GPS / confidence	Permission controls	Confirm retention

Assumptions, Dependencies and Constraints

Assumptions and dependencies: a compatible Android phone and on-device ONNX model are available; camera/location permissions depend on the user/device; internet and API access are needed to synchronize; PostgreSQL is reached through the REST API only. Android device/OS, stakeholder evidence, administrator permissions and location-data retention policy require team verification.

The activity prescribes Flutter Web → REST API → PostgreSQL. Confirm Flutter Web compliance of the web monitor and approval of the additional Android client during review.

Boundary rule

“In scope” means the team commits to analyze, implement, test, and demonstrate the item within the semester. “Out of scope” means it is intentionally excluded from the current version, even if it may be useful in the future. Assumptions are conditions believed to be true; dependencies are external items the system relies on; constraints are limits the team must work within. All four should be explicit to prevent uncontrolled scope growth.

9. Part E - System Use Case Diagram and Use-Case/User-Story Refinement

Model the required behavior from the user's point of view. Every group must first create one complete UML use case diagram for its own approved semester system. The diagram must be based on the actual user roles, functional requirements, scope, and permissions defined in this activity. After the diagram is completed, refine the major use cases and/or user stories in the worksheet below. Teams using a predictive approach may emphasize detailed use-case descriptions, while teams using an adaptive/Agile approach may also maintain user stories and acceptance criteria. The diagram and the written requirements must remain consistent.

Mandatory Use Case Diagram Requirement

Each group must create a UML use case diagram for its own approved system. The diagram is a required Activity 5 output and must be inserted in the SRS or attached as a clearly labeled figure.

- Draw one system boundary rectangle labeled with the official system title.
- Place all major actors or user roles outside the system boundary.
- Show the major user-goal use cases inside the boundary using clear verb-based names (for example: Submit Report, Approve Request, View Map, or Manage Account).
- Connect actors to the use cases they are allowed to perform. Use <<include>> and <<extend>> only when they are logically justified.
- Ensure all Must/High-priority functional features are represented and that the diagram agrees with the role-permission matrix, system scope, and functional requirements.
- Do not place Flutter, the REST API, PostgreSQL, database tables, screens, or other internal implementation components as actors or use cases. These belong to architecture/design artifacts, not the UML use case diagram.
- Required files: readable PNG of the final use case diagram plus the editable source file (for example .drawio, .fig, or equivalent).

RoadWatch System Use Case Diagram

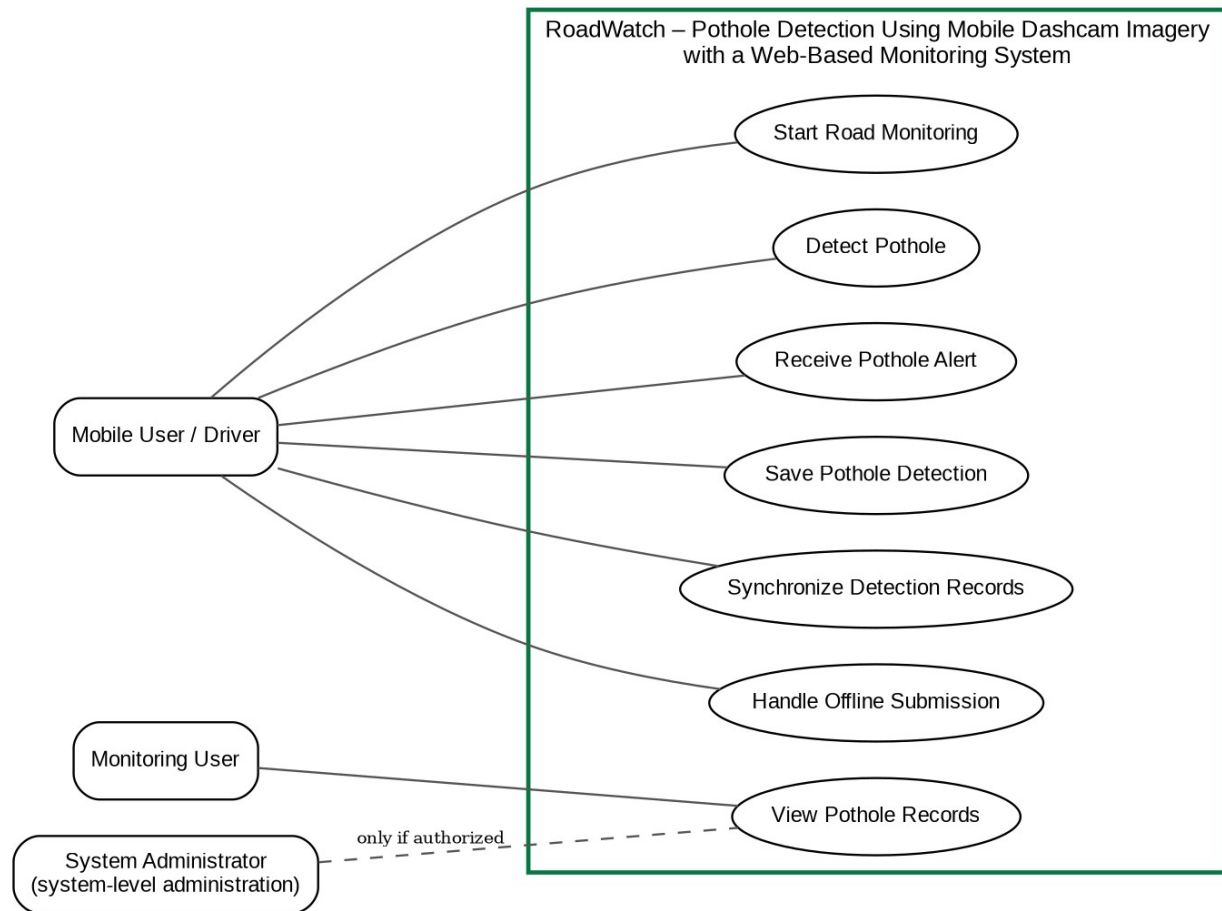


Figure 1. RoadWatch use case diagram. Administrator permissions remain subject to team confirmation.

Use-Case / User-Story Refinement

ID	Actor / User	Goal / Need	Trigger / Input	Main Interaction or User Story	Expected Result / Acceptance
UC-01	Driver	Start monitoring	Start + camera	Start monitoring using camera. If camera permission is denied, show an error and do not start.	Road session runs only with camera access.
UC-02	Driver	Detect pothole	Camera frame	Analyze frames; accept class 3; ignore classes 0–2 for all downstream actions.	Only a pothole proceeds to alert and recording.
UC-03	Driver	Receive alert	Class-3 detection	When a class-3 pothole is accepted, play an audible alert.	A driver can hear the alert.
UC-04	Driver	Save detection	Accepted pothole	Save accepted class-3 detection with GPS if available; do not invent coordinates when GPS is unavailable.	Valid pothole record remains available.
UC-05	Driver	Synchronize	Network returns	Send a queued pothole record to API when network returns; on failure keep it pending.	Successful sync creates no duplicate database row.
UC-06	Monitor	View records	Open web monitor	Open web monitor; retrieve authorized pothole records through API.	Stored pothole records appear in web monitor.
UC-07	Driver	Handle offline	No connection	When offline, retain a valid pending pothole record; retry after reconnection.	Pending record survives until successful sync.

Refinement rule

For a use case, identify the actor, goal, precondition when relevant, main flow, alternative/error flow, and expected postcondition. For a user story, use a form such as “As a [role], I want [capability], so that [value]” and attach acceptance criteria. Every Must/High- priority functional requirement should link to at least one use case, user story, or clearly defined system event.

10. Part F - Feature Priorities, Acceptance Criteria, and Project Constraints

Planning Item	What Must Be Decided	Your Project Decision / Evidence
Feature prioritization	Choose and apply one consistent priority scheme such as Must/Should/Could/Won't-for-this-semester or High/Medium/Low.	Must: F-01–F-05; Should: response-time target; crack reporting: Won't.
Acceptance criteria	Define observable conditions that must be true before a requirement or feature is considered complete.	Pothole class 3 follows alert → save → API → DB → web; cracks do not.
Business rules	Identify approval rules, calculations, validations, status transitions, eligibility rules, or other domain logic.	Allow class 3 only; confidence 0–1; GPS when available; dedupe on retry.
Project constraints	Record deadline, team size, required stack, hardware/device, hosting, data-access, policy, budget, or other fixed limits.	Android phone, YOLO/ONNX, FastAPI, PostgreSQL, semester scope.
Data and privacy constraints	State which data are sensitive, who may access them, retention/sharing limits, and minimum protection needed.	GPS is location-sensitive; restrict web view; retention decision pending.
External-service dependencies	Identify maps/GIS, ML service/model, email/SMS, weather, storage, or other services and what happens when they are unavailable.	On-device model/phone GPS; network and API for sync; queue offline.
Performance / reliability targets	Define realistic response-time, availability, data-consistency, recovery, or workload expectations when relevant.	Proposed alert target: ≤2 s on declared test phone; offline queue survives restart.
Validation and error handling	Define required input validation, user feedback, failed- request handling, and recovery behavior.	Reject bad/class 0–2 payloads; show errors and retain failed queued items.

Feature Priorities and Planned Acceptance Evidence

Feature	Priority	Evidence to collect (not yet completed)
F-01 Monitoring and class-3 detection	Must	Camera session and class-3 test; crack-only zero-downstream test.
F-02 Pothole alert and record	Must	Audible alert, locally saved record, optional GPS check.
F-03 Offline queue and synchronization	Must	Offline save, app restart, reconnection retry, duplicate-protection check.
F-04 API validation and PostgreSQL	Must	Valid class-3 request stored; crack/invalid payload rejected.
F-05 Web pothole review	Must	Authorized monitoring user displays only stored pothole records.
NFR-01 response-time target	Should / proposed	Time 10 accepted detections on declared Android test phone; do not claim achieved yet.

Recommended minimum requirement evidence in the project repository or shared workspace:

docs/requirements/ — SRS, traceability matrix, use-case diagram/source, roles, scope, and review notes. README.md — current baseline/version and status.

11. Part G - Construct the System Requirements Specification

Assemble the completed analysis into one coherent SRS. The document must be project-specific and internally consistent. Do not copy generic requirements from another system. Use stable requirement IDs so changes can be traced during implementation and testing.

No.	Required SRS Element	What Must Be Shown	RoadWatch evidence / location
1	Document control and project identity	Official system title, group, version/date, selected methodology, and prescribed Flutter Web- REST API- PostgreSQL architecture.	Cover; v1.0 draft; Agile/Scrum; prescribed architecture in Part D.
2	Problem and target users	Community/organizational problem, current situation, intended beneficiaries, primary users, and stakeholders.	Part A: project context and stakeholder analysis.
3	System objectives	Specific outcomes the system is intended to achieve without describing unsupported future features.	Part D: system objective.
4	Scope and boundaries	In-scope functions/modules, explicit out-of-scope items, assumptions, dependencies, and constraints.	Part D: scope worksheet and assumptions.
5	User roles and permissions	Role definitions, allowed/restricted actions, data-access scope, and authentication/authorization needs.	Part C: completed role-permission matrix.
6	Functional requirements	Uniquely identified, clear, atomic, feasible, and testable statements of required system behavior.	Part B: FR-01 to FR-14.
7	Non-functional requirements	Measurable quality requirements for relevant areas such as security, performance, usability, reliability, compatibility, maintainability, and deployment.	Part B: NFR-01 to NFR-08.
8	Use cases and/or user stories	Actors/roles, user goals, main interactions or stories, alternative/error handling when relevant, and expected results.	Part E: diagram and UC-01 to UC-07.
9	Features and priorities	Feature list grouped from requirements with a consistent priority scheme and semester- delivery decision.	Part F: F-01 to F-05 and response-time target.
10	Acceptance criteria and business rules	Observable completion conditions, validations, calculations, approvals, transitions, and other domain rules.	Parts B, E and F: planned checks and business rules.
11	Architecture and data alignment	Which requirements are handled by Flutter Web, the REST API/business logic, PostgreSQL, and any approved external services.	Part H: architecture/data mapping; web technology confirmation pending.
12	Traceability and review status	Requirement IDs linked to features/use cases or stories, architecture/data components, acceptance evidence, open issues, and review status.	Parts H and I: all 22 IDs, planned verification and pending review.

Minimum SRS Rule

Every Must/High-priority feature must be supported by one or more functional requirements and any relevant non-functional requirements. Each requirement must have a unique ID, a clear source or rationale, a priority, and a verification/acceptance method. Requirements must remain feasible within the selected methodology, semester scope, and prescribed Flutter Web -> REST API -> PostgreSQL architecture.

12. Reference Example - Do Not Copy as Your Final Output

The example below illustrates the analysis and traceability logic expected in Activity 5. Replace every generic label with the actual community problem, users, requirements, features, roles, constraints, architecture components, and acceptance evidence of your own semester project.

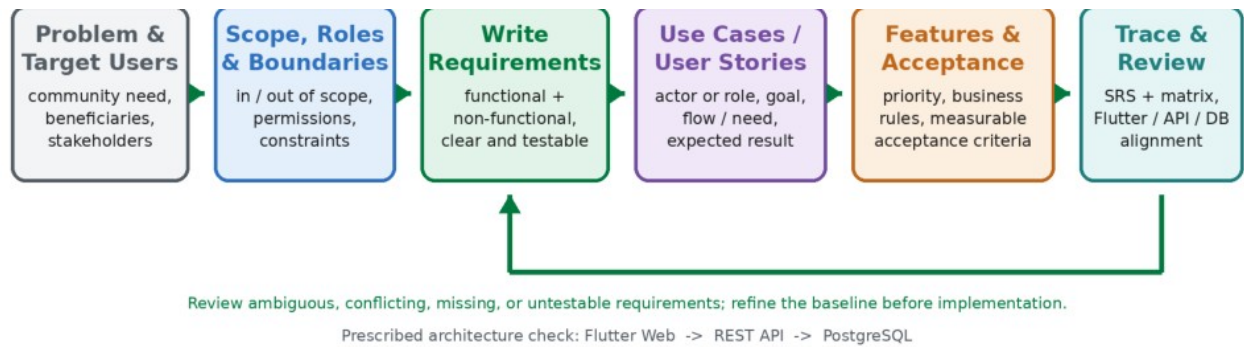


Figure 2. Reference requirements analysis and traceability flow.

Step	Reference Flow
1	Start from the verified community/organizational problem, target users, stakeholders, and requirement evidence.
2	Define system scope, role permissions, boundaries, assumptions, dependencies, and project constraints.
3	Write uniquely identified functional and non-functional requirements that are clear, feasible, and testable.
4	Refine the behavior through use cases and/or user stories that reflect the team's selected development methodology.
5	Group requirements into prioritized features and define measurable acceptance criteria and business rules.
6	Trace requirements to features, Flutter Web/API/PostgreSQL responsibilities, acceptance evidence, and review status; revise gaps before implementation.

13. Part H - Requirements-to-Feature Traceability Matrix

System module / feature set:

RoadWatch pothole monitoring, API storage and web review

Requirements baseline version / date:

v1.0 draft / September 19, 2026

ID	Requirement Summary	Linked Feature & Use Case / Story	Stack Component / Data	Acceptance / Verification & Status
FR-01	start a road-monitoring session using the phone camera.	F-01; UC-01/02	Android camera	Start session; show the camera feed. Planned
FR-02	recognize Pothole (class 3) in camera imagery using the deployed YOLO/ONNX model.	F-01; UC-01/02	Android / ONNX	A class-3 sample is accepted. Planned
FR-03	ignore model classes 0–2 for alert, save, GPS recording, API submission, database storage and web display.	F-01; UC-01/02	Phone filter + API check	Crack-only test produces zero downstream actions. Planned
FR-04	play an audible alert for an accepted pothole detection.	F-02; UC-03/04	Android audio	Sound plays for accepted class-3 detection. Planned
FR-05	attach the phone's GPS coordinates to a pothole record when GPS is available.	F-02; UC-03/04	Phone GPS; API	Saved latitude/longitude match test location. Planned
FR-06	save each accepted pothole detection as a detection record.	F-02; UC-03/04	Phone local record	Saved detection is present in local records. Planned
FR-07	When offline, the app shall queue eligible pothole records locally.	F-03; UC-05/07	Phone local queue	A new accepted record is marked pending offline. Planned

ID	Requirement Summary	Linked Feature & Use Case / Story	Stack Component / Data	Acceptance / Verification & Status
FR-08	When connectivity returns, the app shall retry synchronization of queued records.	F-03; UC-05/07	Phone queue; API	Pending record is retried after reconnection. Planned
FR-09	submit eligible pothole records through the REST API.	F-03; UC-05/07	Android → FastAPI	An accepted request reaches FastAPI. Planned
FR-10	store valid pothole detection records in PostgreSQL.	F-04; UC-05	FastAPI → PostgreSQL	Corresponding record exists in the database. Planned
FR-11	The web monitoring system shall display stored pothole records.	F-05; UC-06	Web → API → database	A stored record appears in the web monitor. Planned
FR-12	reject invalid and non-pothole detection payloads.	F-04; UC-05	FastAPI validation	Crack/invalid payload is not stored. Planned
FR-13	The system shall prevent retries of the same queued record from creating duplicate database rows.	F-03; UC-05/07	Client ID; API; DB	Retry same record ID; database has one row. Planned
FR-14	The monitoring user shall be able to review available pothole records through the web monitoring system.	F-05; UC-06	Web → API	Monitoring user opens available records. Planned
NFR-01	On the declared test phone, audible alert should begin within 2 seconds after an accepted pothole detection.	F-02; UC-03	Android / audio	Time 10 accepted detections; record measurements. Planned
NFR-02	A submitted detection confidence shall be numeric and within 0–1 inclusive.	F-04; UC-05	API input validation	Test 0, 1, out-of-range and nonnumeric values. Planned
NFR-03	Queued pothole records shall remain available after the app closes and reopens while offline.	F-03; UC-07	Android local queue	Queue survives offline app restart. Planned
NFR-04	When available, saved latitude shall be within –90 to 90 and longitude within –180 to 180 and shall not be swapped.	F-02/F-04; UC-04	GPS; API; DB	Test known coordinates and bounds. Planned
NFR-05	run on the team's declared Android test device and OS version.	F-01; UC-01/02	Android test phone	Record Android device/OS and test result. Planned
NFR-06	Mobile and web clients shall access stored detection data through the REST API, not directly from the database.	F-03/F-04/F-05; UC-05/06	Client/API/DB layers	Check data flow/configuration. Planned
NFR-07	give understandable status/error feedback for camera denial, GPS unavailability and failed submission without falsely showing success.	F-01/F-03; UC-01/07	Android UI; API	Simulate failures and inspect on-screen status. Planned
NFR-08	The Android app shall not connect directly to PostgreSQL or contain PostgreSQL connection credentials.	F-03/F-04; UC-05	Android and API config	Inspect app configuration and network route. Planned

14. Part I - Requirements Review and Baseline Approval

Before submission, conduct a structured group review and prepare the SRS for the online requirements review. Record issues that must be revised instead of silently changing the baseline. Confirm all of the following:

- Every requirement is necessary, understandable, uniquely identified, feasible within the semester, and testable or verifiable.
- Functional requirements, non-functional requirements, roles/permissions, scope, use cases/user stories, features, priorities, and acceptance criteria agree with one another.
- No Must/High-priority feature lacks supporting requirements or acceptance evidence, and no requirement is orphaned from a feature, user need, or system-quality objective.
- The requirements are consistent with the selected development methodology and with the prescribed Flutter Web-REST API-PostgreSQL architecture.
- Ambiguous, conflicting, duplicate, missing, or out-of-scope requirements are identified and assigned a revision decision before implementation begins.

Online review status: Requires revision / group verification before instructor review.

Reviewer / consultation source: Pending group verification and instructor online review.

Review date: Pending. Baseline version: v1.0 draft, September 19, 2026.

Most important revision to resolve: Confirm administrator permissions and Flutter Web compliance of the web monitor, including approval of the additional Android client.

Baseline v1.0 is a draft for group verification and instructor online review. No completed stakeholder interview, performed acceptance test, approved baseline or instructor review is claimed in this document. After comments, record changes, update any affected requirements, and revise this baseline under Agile/Scrum.

Documentation repository: <https://github.com/cyrill2026/RoadWatch> – activity documentation only. Place the SRS, matrix, use-case PNG/editable source and supporting notes under docs/requirements/; do not upload the full RoadWatch implementation source code there.

15. Final Quality Checklist

Checklist A	Checklist B
☐ The community/organizational problem and target users are clearly stated and supported by project evidence.	☐ Every requirement has a unique ID and uses project-specific, understandable wording.
☐ Functional requirements describe required behavior rather than vague features or implementation details.	☐ Non-functional requirements are measurable or verifiable where practical.
☐ User roles, permissions, restricted actions, and data-access boundaries are consistent across the SRS.	☐ In-scope and out-of-scope items are explicit; assumptions, dependencies, and constraints are documented.
☐ Use cases and/or user stories are linked to actual roles and required system behavior.	☐ Features are prioritized consistently and acceptance criteria are observable and testable.
☐ Business rules, validation, error handling, and external-service dependencies are documented when relevant.	☐ Requirements are aligned with Flutter Web, the REST API, PostgreSQL, and approved external services.
☐ The requirements-to-feature matrix contains no unexplained orphan requirements or unsupported Must/High features.	☐ The SRS and traceability matrix agree with the selected development methodology and semester plan.
☐ The requirements baseline has been reviewed for ambiguity, conflict, duplication, feasibility, and scope creep.	☐ Every group member can explain assigned requirements, role permissions, acceptance criteria, and traceability decisions during consultation or review.

16. Submission Requirements

- Final System Requirements Specification in editable DOCX and PDF format, using the team's current requirements baseline/version.
- Completed Requirements-to-Feature Traceability Matrix in the SRS or as an accompanying spreadsheet/table file.
- Completed role-permission matrix, scope/boundary worksheet, use-case diagram and/or user-story set, feature-priority list, and acceptance-criteria evidence.
- Updated project GitHub repository or shared workspace containing the requirement artifacts under a folder such as docs/requirements/.
- Repository link and required files submitted through MasaoLMS using the naming convention IT110_LA5_SystemName.
- Complete the online requirements review. If the instructor returns comments or identifies requirement defects, submit the revised baseline and preserve a brief record of the changes made.

Academic Integrity and Competency Validation

The SRS must represent the team's own approved project, stakeholder evidence, scope decisions, and technical constraints. AI tools may be used to clarify terminology, refine wording, or check consistency, but generated requirements must not be accepted without verification against the team's actual system. Every member is responsible for understanding the requirements assigned to the group. The instructor may ask any member to explain the source, purpose, priority, permission rule, acceptance criterion, or implementation layer of a requirement without assistance.

17. Scoring Rubric - 80 Points

Partial credit may be awarded when evidence is present but incomplete, ambiguous, inconsistent, untestable, outside the approved scope, or not traceable to the team's actual project. Requirements returned during the online review should be revised before they are used as an implementation baseline.

Criterion	Evidence for Full Credit	Points
Problem, target users, and stakeholder analysis	The SRS clearly identifies the real community/organizational problem, current situation, target users, stakeholders, goals, and evidence used to derive requirements.	8
Functional requirements	Functional requirements are complete for the approved scope, uniquely identified, project-specific, atomic, feasible, clear, prioritized, and testable.	14
Non-functional requirements	Relevant quality requirements are measurable/verifiable and appropriately address security, performance, usability, reliability, compatibility, maintainability, deployment, or other project needs.	10
User roles, permissions, and scope boundaries	Roles, allowed/restricted actions, data access, in-scope/out-of-scope items, assumptions, dependencies, and constraints are accurate and mutually consistent.	10
Use cases/user stories and feature priorities	Use cases and/or user stories correctly represent user goals and required behavior; features are logically grouped, linked to requirements, and prioritized for the semester.	10
Acceptance criteria, business rules, and constraints	Acceptance conditions are observable/testable; validation, calculations, approvals, errors, external dependencies, and important project constraints are documented when relevant.	8
Requirements-to-feature traceability and architecture alignment	The matrix links requirements to features/use cases or stories, Flutter Web/API/PostgreSQL responsibilities or data, and acceptance evidence without unexplained gaps or contradictions.	12
SRS quality and online requirements review	The document is organized, readable, versioned, internally consistent, submitted correctly, and reviewed for ambiguity, duplication, conflict, feasibility, traceability, and scope before implementation.	8

Raw Score	Equivalent	Checked by	Date
/ 80	_____	_____	_____

18. Post-Laboratory Reflection

Which requirement was the most difficult to make clear and testable, and what did your group change?

The most difficult requirement was FR-03. We made it testable by checking that crack detections create no alert, saved record, GPS record, API submission, database row or dashboard entry.

Which requested feature or capability was placed out of scope for this semester, and why?

Crack reporting remains out of scope because the current application is pothole-only. Classes 0–2 must not trigger downstream actions.

Which user role has the most security-sensitive permissions, and what control should protect those actions?

The system administrator has the most security-sensitive access. System-level actions should be restricted to authorized administrators; exact administrative permissions still require team confirmation.

Which non-functional requirement has the greatest effect on your Flutter Web-REST API-PostgreSQL architecture?

NFR-08 has a major architecture effect because the mobile client must submit through the REST API instead of connecting directly to PostgreSQL or containing database credentials.

What requirement changed or became clearer after the requirements review, and how will your selected methodology manage that change?

Instructor review has not happened yet, so no review-driven change is claimed. If requirements change, we will update the Scrum backlog, traceability matrix and requirements baseline.